

Podman Technical Overview

Podman (short for *Pod Manager*) is an open source, OCI-compliant container engine that provides a Docker-compatible CLI for managing containers, images, volumes, and pods ¹. Unlike Docker, Podman runs in a *daemonless* architecture – there is no central `dockerd` service. Instead, each container is launched as an individual process (child of the invoking user's session) with Podman using lightweight wrapper processes (like **Conmon**) to monitor container lifecycles ² ³. This design enables Podman to run containers as regular unprivileged users (rootless mode) by default, enhancing security and reducing overhead ⁴ ⁵. Podman was originally developed by Red Hat as part of the libpod library (and initially tied to the CRI-O project) to offer a Docker-like experience without a daemon ⁶. It has since grown into a full-featured container engine, now maintained in the open under an Apache 2.0 license ⁷ and accepted into the Cloud Native Computing Foundation (CNCF) as a Sandbox project in 2025 ⁸.

Advantages of Podman over Docker/Containerd

Daemonless Architecture: Podman's most significant advantage is its lack of a long-running daemon. Each `podman run` launches the container directly as a child process of the user, rather than routing through a root-owned service ². This eliminates the single point of failure and idle resource usage of the Docker daemon ⁹. If one container process crashes, it doesn't bring down a whole service – other containers keep running independently ⁹. There's also no need to start or manage a background service; Podman commands operate directly on the OS.

Rootless Security: Podman was designed "security-first" to run containers without root privileges by default ¹⁰ ⁵. It uses Linux user namespaces to map container root (UID 0) to an unprivileged UID on the host, so even if a process breaks out of the container, it only has the permissions of a normal user ¹¹. In contrast, Docker normally requires a root daemon (or adding users to the `docker` group, which is effectively root access) for container management ¹² ¹³. While Docker has a rootless mode, it's an opt-in feature with limitations and not widely adopted ¹⁴. Podman's out-of-the-box rootless operation means **no background root process** (`/var/run/docker.sock`) that could be exploited ⁹ ⁵. This dramatically reduces the attack surface – Podman has no privileged socket for containers to abuse, and no central daemon for attackers to target ¹⁵ ⁵. Podman also drops unnecessary Linux capabilities by default (11 caps vs Docker's 14) and integrates with SELinux/AppArmor for confinement if available ⁵ ¹⁶. The result is a more secure runtime by design.

CLI Compatibility and Ease of Transition: Podman deliberately mirrors the Docker CLI syntax and image formats, making migration simple. Most Docker commands work the same with Podman (e.g. `podman run`, `podman build`) and one can even alias `docker=podman` in the shell ¹⁷ ¹⁸. Podman uses the same OCI image standards and can pull images from Docker Hub or any registry. It also provides a `podman-docker` wrapper (or package) that installs a `docker` command linked to Podman, so existing scripts using `docker` continue to function ¹⁹ ²⁰. In practice, Podman offers ~95% CLI compatibility for common use cases, with only minor differences (e.g. in volume handling or networking defaults) ²¹. This means teams can switch to Podman with minimal retraining – an important advantage for organizations concerned about Docker's licensing changes or wanting a drop-in replacement ²² ²³.

Pods and Kubernetes Integration: Podman extends the concept of pods (groups of containers sharing resources) from Kubernetes into local container management. It natively supports running multiple containers in a pod, which Docker does not do outside of Docker Compose or Swarm. This makes Podman especially attractive for Kubernetes users: you can define a pod locally with `podman pod create` or run several services together, then generate Kubernetes YAML directly from your Podman containers using `podman generate kube` ²⁴. Likewise, Podman can **play** Kubernetes manifests (`podman play kube`) to run them on a single host ²⁴. This tight integration provides a *frictionless dev→prod workflow* – developers can prototype with Podman and easily transition those workloads to Kubernetes without rewriting definitions ²⁴. Containerd, by contrast, is primarily a low-level runtime for Kubernetes and doesn't offer a user-facing CLI or comparable pod management features (tools like `nerdctl` add a Docker-compatible CLI to containerd, but still lack Podman's easy pod support) ²⁵ ²⁶. In summary, Podman's built-in pod support and Kubernetes-friendly tooling make it a better choice for cloud-native development.

No Docker Daemon Overhead: Because Podman has no daemon, it consumes fewer system resources when idle. Docker's `dockerd` typically uses ~150MB RAM even with no containers running, whereas Podman's footprint is near-zero when not in use (each container's overhead is just the small Common monitor process, around ~50MB per container) ²⁷. This can improve density on multi-tenant systems and reduce noise in development environments. Podman containers also tend to start up a bit faster on first use since there's no need to communicate with a daemon process – though subsequent startups in Docker may even out due to caching, real-world tests often show Podman slightly ahead in startup time and using less memory overall ²⁷ ²⁸. Containerd is lightweight on its own, but it's not as usable directly by end-users; Podman combines a lightweight runtime with a full-featured user interface.

Systemd Integration: Podman plays nicely with init systems and Linux process management. In environments with **systemd**, Podman can utilize systemd as its cgroups manager and can even have systemd supervise container processes. By default, Podman on a systemd-based distro will create cgroups under the user's slice (or root slice for rootful containers) using the **systemd cgroup driver** ²⁹. This means you can see and control Podman-launched containers via systemd like regular services (each container gets a scope unit when run as a systemd-managed service). Moreover, Podman provides a convenient way to generate systemd unit files for containers – the command `podman generate systemd` (or using **Quadlet** files) will create a unit that ensures a specific container (or pod) starts on boot and is monitored by systemd ³⁰. Administrators appreciate this because it eliminates the need for workarounds (like writing custom systemd units to call `docker run`). Docker requires an always-running daemon, but Podman containers can be launched at boot via the normal init system, fitting naturally into server management workflows ². On non-systemd systems, Podman can fall back to managing cgroups via **cgroupfs** directly ²⁹, and use its own service (`podman system service`) if an API or remote access is needed. In Alpine Linux (which uses OpenRC), for example, Podman ships an init script (`podman-openrc` package) to optionally run a Podman REST API service at boot for remote client usage ³¹ ³². Overall, Podman's architecture aligns with traditional Unix principles – no special daemon, leverage existing init and control mechanisms – which can simplify operations and increase reliability ³³ ³⁴.

Comparison with Containerd: Unlike Docker, *containerd* is not a full container platform but a core runtime service (used under the hood by Docker and Kubernetes). Containerd excels at running containers in Kubernetes or as a component of a larger system, but it doesn't provide a user-friendly CLI or tools for building images (one usually pairs it with **BuildKit/nerdctl** for those capabilities). Podman, on the other hand, is a complete solution for managing containers and images on a single host. It includes the ability to build images (via Buildah integration) ³⁵, push to registries (with Skopeo libraries), and run containers interactively or in the background – all without needing Docker or

containerd. In fact, Podman can fulfill the role of Docker Engine for most use-cases, while using standard OCI runtimes under the hood (such as **crun** or **runc**). In practice, containerd is often chosen for production Kubernetes clusters or embedded scenarios, whereas Podman is chosen for its user-facing features and security in standalone environments. As one 2025 comparison puts it, use Docker when you need broad ecosystem and familiarity, containerd for low-level Kubernetes performance, but **“choose Podman when security is a top priority, you need rootless containers by default, and want Docker compatibility without the daemon overhead”** ²⁶ ³⁶ .

Common and Advanced Use Cases

Podman’s feature set lends itself to several scenarios, from everyday development to specialized enterprise use cases:

- **Developer Workstations and CI Pipelines:** Podman is a drop-in replacement for Docker on Linux dev machines. Developers can run containers without needing root access or special daemon setup, which is especially useful in corporate environments with strict privilege policies. In continuous integration (CI) systems, Podman enables safer parallel builds: each build job can spawn containers rootlessly in its own user namespace, avoiding the security risk of all jobs sharing a root-level Docker daemon ³⁷ . For instance, GitLab CI runners or Jenkins agents can use Podman so that one job’s containers cannot interfere with another’s (no more accidentally exposing the Docker socket to containers). This isolation is a game-changer for multi-tenant CI infrastructures ³⁷ .
- **Multi-User Servers and HPC:** On shared servers (such as academic clusters or HPC environments), giving every user access to Docker can be problematic because the Docker daemon effectively grants root-equivalent powers. Podman provides a solution: users can run **rootless containers under their own UID**, which respects multi-user security. System admins don’t need to maintain a daemon or worry about container breakout as much, since each user’s containers run with that user’s privileges. This model aligns with HPC scheduling as well – users can run containerized workloads (even GPU-accelerated, with proper driver access) without root, making Podman popular in research and high-performance computing circles. It’s essentially containerization without compromising the host’s security model.
- **Regulated and Secure Environments:** Industries like finance, healthcare, and government have strict compliance rules (least privilege, auditing, no persistent elevated services). Podman’s architecture directly caters to these needs ³⁸ . There is no always-root process to violate least-privilege principles, and actions can be logged via the system journal or standard auditing. Each container can be tied to a systemd unit (with logging), satisfying audit requirements for traceability ³⁹ . Furthermore, Podman’s use of user namespaces and SELinux/AppArmor means it can meet hardened security profiles out of the box. Enterprises that disallow Docker for security reasons often approve Podman as it adheres to kernel security features more strictly. In fact, Podman is the default container engine in Red Hat Enterprise Linux and related distributions, underlining its role in security-focused deployments ⁴⁰ ⁴¹ .
- **Dockerless Servers and Edge Computing:** Podman is well-suited for minimal or embedded systems where running a full Docker daemon is undesirable. Its lightweight nature (no daemon process) and tolerance for non-root operation make it a good fit for edge devices or IoT gateways running Alpine Linux, Fedora IoT, etc. An admin can script Podman to start necessary containers at boot via init scripts or systemd units, without introducing a large attack surface. For example, one could run containerized microservices on a remote edge server with Podman,

managed through SSH (no API socket exposed unless needed). Podman's minimal dependencies (it can run with just runc/crun and kernel features) and smaller footprint are advantageous in resource-constrained environments.

- **Bridging Development to Kubernetes:** Many teams use Podman on developers' laptops and staging servers to emulate Kubernetes behavior. With Podman's pod concept and the `generate kube` feature, a developer can define a multi-container setup and then export it as Kubernetes YAML when it's ready for production ²⁴. Conversely, existing Kubernetes YAML configurations can be run on a single machine using `podman play kube` for testing. This two-way compatibility is an advanced use case that significantly simplifies testing of k8s deployments locally. As an example, a developer could run a database and app server as a Podman pod on their machine, verify everything, then *generate* the Kubernetes spec and commit it to the repo – reducing environment drift between local and cluster. Docker lacks this capability natively (Docker Compose can define multi-container apps, but converting Compose files to Kubernetes manifests is a separate process). Thus, organizations heavily into Kubernetes often standardize on Podman for development to streamline their workflow ⁴² ⁴³.
- **Avoiding Docker Licensing and Vendor Lock-in:** Since Docker Desktop introduced licensing fees for larger businesses (and Docker Engine's future is uncertain in some respects), companies have been motivated to find alternatives. Podman is entirely open source (Apache 2.0) with no licensing costs or usage tracking ⁴⁴ ⁴⁵. Teams of hundreds of developers can switch to Podman (with Podman Desktop as a free GUI if needed) and avoid hefty subscription fees. The **Podman Desktop** GUI, while not as mature as Docker Desktop yet, is improving rapidly and is also freely available ⁴⁶ ⁴⁷. This cost factor, combined with support from Red Hat and now CNCF, gives many organizations confidence in adopting Podman as a long-term solution. In practical terms, migrating from Docker to Podman can be as easy as installing Podman and updating scripts or environment variables to point to Podman's socket or CLI – the transition is often seamless (e.g., one can set `alias docker=podman` and continue using existing Docker-based tooling in many cases) ⁴⁸ ¹⁸. This low switching barrier and freedom from licensing make Podman attractive for businesses trying to reduce expenses or avoid vendor lock-in in their container strategy ²³.
- **Podman-Compose and Container Orchestration:** While Docker has docker-compose and Swarm for multi-container orchestration, Podman supports similar workflows through *podman-compose* (a Python tool) or its own Compose compatibility built-in. Advanced users can deploy Compose YAML files with `podman compose up` (supported since Podman 4.x) ⁴⁹, achieving roughly 85–90% coverage of Docker Compose features ⁴⁹. This is useful for local development or single-host deployments where using Kubernetes might be overkill. For example, a small team could use Podman Compose to spin up a stack of services on a VM as a simpler alternative to full Kubernetes. Additionally, Podman's integration with systemd means it can handle what some might use Docker Swarm or Compose for – you can define dependencies and auto-restart policies via systemd units for each container. While Podman doesn't aim to replace full orchestrators beyond a single node, these capabilities cover many "advanced" use cases that previously kept people tied to Docker.

In summary, Podman excels in scenarios where **security**, **rootless isolation**, and **Kubernetes alignment** are important. It allows container use in environments that couldn't tolerate a root daemon (shared servers, CI, tightly secured systems) and provides tools that smooth the path to cloud-native infrastructure. Docker still has strengths in areas like a broader ecosystem, more mature GUI tooling on Mac/Windows, and familiarity for users – but Podman's use cases are rapidly expanding as its feature set and community grow.

Technical Architecture Overview

Daemonless Design: Podman's architecture is often described as *fork-exec* model. When you run a Podman command (e.g. `podman run -d nginx`), the Podman CLI process itself forks off the container process directly, without intermediation by a service daemon ². Under the hood, Podman uses a helper called **Common** (Container Monitor) to set up and track the container process. Common is a tiny program that launches the OCI runtime (like runc or crun) and then sits in the background to monitor the container's STDIO and exit status ⁵⁰. Each container gets its own Common process (and PID namespace leader), but there is no single "Podman daemon" holding state. The container's life cycle is tied to that Common + container process group. This means if you close the Podman client or log out, your containers remain running (their parent is re-parented to init or managed by systemd – see below). To list or manage containers, Podman reads their state from a SQLite database and the OCI runtime (no daemon memory needed). The **trade-off** of no daemon is a slight overhead on each new container start (Podman must initialize the storage and network for each run anew). The first Podman use after a reboot can be ~10% slower to start a container than Docker's (since Docker's daemon pre-warms these things), but subsequent runs are fast, and this gap has narrowed with optimizations ⁵¹. There's also a shift in how you monitor containers: with Docker, you watch the one daemon process; with Podman, you might use tools like `ps` or `systemd` to see multiple container processes ⁵². However, this is largely mitigated by `systemd` integration – on a `systemd` system, each Podman container can appear as a service scope, making it clear and manageable.

Rootless Containers via User Namespaces: A cornerstone of Podman's architecture is user namespaces, which allow unprivileged users to create container environments where they *appear* to be root inside the container. When running rootless, Podman automatically requests a range of subordinate UIDs/GIDs (as configured in `/etc/subuid` and `/etc/subgid`) for the user to map inside the namespace ⁵³. For example, if the user has `100000-165535` subuids, Podman can map UID 0 in the container to host UID 100000. All files and processes in the container that show as owned by "root" are actually owned by UID 100000 on the host. The Linux kernel ensures isolation, so those IDs have no special privileges on the host ¹¹. Podman will automatically set up these mappings if the mappings are present; the administrator's responsibility is to allocate them (often done by the OS during user creation, or manually by editing `/etc/subuid`). Additionally, Podman uses the **newuidmap/newgidmap** helpers (from the `shadow-subids` package in Alpine) to securely configure these mappings ⁵⁴. Thanks to user namespaces, Podman can drop *all* root requirements for container processes – the container's "root" is just a regular user from the host's perspective ⁵⁵. This enables powerful features like multiple users running containers concurrently without interfering with each other, and it significantly limits the impact of a container breakout (an attacker who becomes root inside only gets a fake root, mapped to a non-root UID outside ⁵⁵). It's worth noting that Docker's experimental rootless mode also uses user namespaces, but Podman's implementation has been native from the start and doesn't require special setup scripts or daemon flags ¹⁴. Podman rootless supports most container features, though some (e.g. listening on low host ports or mounting certain filesystems) have limitations due to kernel restrictions – these can often be worked around by tuning `sysctls` (like allowing unprivileged users to bind ports <1024) or using `fuse-overlayfs` for rootless storage. Overall, user namespaces are the "secret sauce" that allow Podman to be both secure and convenient.

OCI Runtime and Cgroups: Podman is *not* a replacement for low-level runtimes like runc; rather it orchestrates them. When you launch a container, Podman delegates to an OCI-compliant runtime to actually set up the container in the kernel. By default, Alpine's Podman will pull in **crun** (a fast OCI runtime written in C) or **runc** as the `oci-runtime` dependency ⁵⁶ ⁵⁷. Crun is often favored for rootless containers because it has better support for cgroups v2 and is very lightweight. Indeed, the Alpine community recommends using crun to avoid permission issues in rootless mode ⁵⁸. In practice,

after Podman sets up namespaces and filesystem mounts, it invokes the OCI runtime to execute the container process. The runtime (crun/runc) performs the `clone` and `exec` of the user's container command under the configured namespaces, then exits, handing control to the container process (with Conmon monitoring).

For resource limits and accounting, Podman relies on **cgroups**. Modern Podman uses **cgroup v2 (unified)** by default, which is now standard on most distros (including Alpine 3.14+ via openrc's `cgroups` service) ⁵⁹. Podman can operate with cgroup v1 if absolutely needed, but rootless mode cannot limit resources on cgroup v1 due to kernel limitations ²⁹. Thus, enabling cgroup v2 is usually a prerequisite for full Podman functionality (on Alpine, enabling the `cgroups` service sets up a unified cgroup hierarchy) ⁵⁹. Podman's **cgroup manager** can be configured to `systemd` or `cgroupfs`. On systemd-based systems, it defaults to `systemd`, meaning Podman will create cgroup slices and scopes that integrate with systemd's hierarchy ²⁹. On Alpine (OpenRC), Podman defaults to the `cgroupfs` manager (directly manipulating `/sys/fs/cgroup`) since systemd is not available. In either case, Podman ensures each container gets its own cgroup for resource tracking. If you set CPU or memory limits via Podman (`--cpus`, `--memory` flags), it writes those to the container's cgroup just as Docker would. A notable difference is that in rootless mode, cgroup delegation requires some setup: the user's systemd session or cgroup namespace must allow the user to create sub-cgroups. On systemd systems, login managers can delegate a cgroup subtree to the user session. On OpenRC, one may need to run Podman in a cgroup-aware context or simply run without limits (since an unprivileged user can't create cgroup controllers by default on cgroupfs). SUSE's documentation notes that using cgroups v2 and systemd user units is recommended for rootless Podman to handle limits properly ⁶⁰. These details aside, Podman's use of cgroups is standard – it leverages kernel features to enforce limits and account usage, just without a persistent daemon holding the cgroup open (Conmon and the container process hold membership in the cgroup instead).

Networking: Podman supports both rootful and rootless container networking. In rootful mode, it can use the **CNI (Container Network Interface)** plugins similar to Docker (bridge networks, NAT, port forwarding) or the newer *Netavark* network stack (a modern networking backend written in Rust). Alpine's Podman package includes Netavark and its DNS helper Aardvark by default ⁶¹. By default, Podman will create a `podman` bridge network (often `cni-podman0`) for containers, analogous to Docker's `docker0` bridge. For rootless operation, Podman cannot create real network interfaces or modify iptables without privileges, so it uses userland networking: historically **slirp4netns** (SLiRP) was used to allow outbound connectivity from rootless containers. Newer Podman versions have introduced **passt** (a faster user-space networking tool) as the default on systems that support it ⁶². Rootless networking means that containers run by an unprivileged user get NATed networking without needing root, albeit with some limitations (e.g. no inbound ports below 1024 unless user has specific kernel permissions). If a rootless container needs to accept incoming connections, Podman can utilize a VPN kit or ask the user to map high ports or use the `--net=host` mode for trusted scenarios. Networking setup is handled at container start – Podman will set up firewall rules (using `iptables/nftables` through CNI) when run as root, or coordinate slirp4netns/passt processes when run rootless. One important aspect: since there's no daemon, the networking is configured by the Podman CLI process at container start and torn down when the container stops. If all containers are stopped, the `cni-podman0` interface may go away. This is mostly transparent to users, but it's a difference from Docker where the daemon continuously manages the network bridge.

Systemd and Podman Services: As mentioned, on systemd systems, Podman can integrate tightly. There is a **Podman API service** (`podman.socket` and `podman.service`) which can run on demand to serve Docker-compatible API requests. When enabled (via `podman system service --time=0` or the packaged unit), Podman behaves somewhat like the Docker daemon to provide a REST API, for

compatibility with tools that expect Docker's API (e.g. `docker-compose` v2, certain IDE integrations). In Alpine, an OpenRC init script is provided (`/etc/init.d/podman`) which calls `podman system service` in the background ³¹ . This is optional – if you don't need remote API or Docker socket compatibility, you can ignore it. But it's there to allow Podman to act as a **daemon when necessary** (for example, you could run `podman system service` at boot and point Docker clients to `unix:///run/podman/podman.sock`). In day-to-day local use, most users just call the Podman CLI directly and don't run a persistent service.

Storage: Podman and Docker share the same image format and storage drivers (thanks to underlying libraries like containers/storage). By default, Podman on Linux uses the **OverlayFS** storage driver for images and container layers (storing data under `/var/lib/containers` for root, or under `$HOME/.local/share/containers` for rootless). Alpine's default is overlayfs as well ⁶³ . If the backing filesystem doesn't support overlay (like an XFS without `ftype=1` , or if running Podman *inside* a container where overlay can't be nested), Podman can automatically fall back to alternatives like **fuse-overlayfs** (which allows unprivileged overlay) or use a plain **vfs** (copy-on-write directory) storage. The Alpine wiki notes that if you are running Podman inside a Docker container or on a filesystem like Btrfs, you should switch the driver to `vfs` or `btrfs` respectively for stability ⁶³ . This can be done by editing `/etc/containers/storage.conf` . In rootless mode, Podman also uses fuse-overlay by default if kernel overlay is not available. Additionally, because Podman shares image storage with Buildah (its image-building tool), images pulled or built with one tool are immediately available to the other ³⁵ . There is no separate Docker daemon image cache – it's all just files on disk, managed by the storage library. This means you can use `podman pull` to get an image or even use Docker to pull an image (to Docker's storage) and *in some cases* Podman can read it if configured to look at the same storage, but typically Docker and Podman maintain separate stores.

In summary, Podman's architecture comprises several layers: the Podman CLI/tooling, the Conmon monitor, the OCI runtime (runc/crun), and kernel features (namespaces, cgroups, SELinux) – assembled in a daemonless fashion. By leveraging user namespaces and systemd integration, Podman achieves **rootless containers with near parity to rootful ones** in functionality ¹⁰ ⁶⁴ . The design trades a bit of complexity (multiple processes rather than one daemon) for significant gains in security and flexibility. This architecture is considered more robust by some, as it removes a central failure point and aligns container processes with normal OS process management ³³ . It's a different philosophy from Docker's all-in-one daemon, but one that is increasingly validated by use in enterprise Linux distributions and cloud-native workflows.

Open Source Project Governance and History

Podman is a fully open-source project, licensed under Apache License 2.0 ⁷ . It was created by Red Hat engineers around 2017-2018 as part of Red Hat's "containers toolkit" initiative (alongside Buildah and Skopeo) to improve container security and developer tooling. In the early days it was sometimes referred to as **"kpod"** and was originally developed within the CRI-O project context ⁶ (CRI-O is a Kubernetes runtime; Podman was the CLI to run standalone containers using the same libraries). Podman 1.0 was released in early 2019 ⁶⁵ , and since then it has rapidly evolved with a roughly monthly release cadence. Major contributions have come from Red Hat, but also from the broader community (engineers from SUSE, Intel, and independent contributors). As of late 2025, the project had ~800 contributors on GitHub and ~30k stars, indicating a vibrant community ⁶⁶ .

In **governance structure**, Podman (and its related tools) were originally under Red Hat's stewardship – maintainers were largely Red Hat employees. For example, core maintainers include Red Hat engineers like Brent Baude, Nalin Dahyabhai, Daniel J Walsh, etc., who have led development and reviews (as listed

in the project's MAINTAINERS files) ⁶⁷. Red Hat used Podman as the container engine in RHEL 8+ and Fedora, ensuring company support and resources for the project. In October 2024, at KubeCon North America, Red Hat announced it would donate the *Podman project (Podman CLI and Podman Desktop)* to the **Cloud Native Computing Foundation (CNCF)** to foster an open, vendor-neutral future ⁸. The donation was completed in January 2025 when Podman was accepted as a CNCF Sandbox project ⁸. This move signaled that Podman's governance is now overseen by the CNCF Technical Oversight Committee and an open community process, rather than purely by Red Hat. The project's name in CNCF is "*Podman Container Tools*", and it encompasses Podman, Podman Desktop, and related libraries ⁶⁶.

Under CNCF, Podman is expected to follow CNCF's contributor ladder and Code of Conduct. There is a Podman-specific governance doc outlining roles like Core Maintainers, who have commit and release authority, and the process for gaining such roles (typically through sustained contributions and election by existing maintainers). Red Hat remains a major contributor, but other organizations can more easily participate in decision-making now. For instance, SUSE has adopted Podman in SUSE Linux Enterprise, and we see SUSE engineers contributing and joining discussions. The maintainers communicate via the `containers/podman` GitHub repository and mailing lists (which have moved to CNCF infrastructure post-donation). The CNCF Sandbox status indicates early stages in CNCF; as the project matures, it could move to Incubation with evidence of growing adoption and governance diversity.

Regarding **hierarchy and affiliation**, Podman is part of the broader **Containers Organization** on GitHub (`github.com/containers`), which houses related projects like Buildah (image building), Skopeo (registry interactions), CRI-O (Kubernetes runtime), and more. Many of these tools share code (for example, `libpod` and `containers/common` are libraries used by Podman, Buildah, CRI-O). The community tends to overlap, and there's even a weekly or bi-weekly community meeting open to contributors and users (the Podman community meetings, minutes of which are often posted on GitHub). Decisions on major changes (like new features or deprecations) are made through design proposals on the mailing list or issues, reviewed by maintainers and discussed in the open. With CNCF, Podman also inherits the CNCF's neutrality requirements – it will have a project charter and governing board if it advances stages, ensuring no single company can revoke access or change the license, etc.

It's worth noting that Podman Desktop (a graphical management UI similar to Docker Desktop) is a sub-project that also joined CNCF sandbox alongside Podman CLI ⁶⁸. Podman Desktop has its own maintainers and community but aligns with the Podman CLI project goals. Both are under the CNCF "container runtime" umbrella.

In summary, **Podman's governance** has transitioned from a primarily Red Hat-driven project to a more community-driven model under CNCF oversight. The project maintainers include long-time experts from Red Hat (who continue to guide the project's technical direction) as well as growing representation from other organizations. The open source nature is strong: all development happens in public on GitHub, issues and pull requests are welcome from anyone, and releases are Apache-licensed (allowing free use and distribution, including in commercial products) ⁶⁶ ⁷. Podman's inclusion in CNCF also reassures users of its long-term viability and openness. Users can find the project website at **podman.io** with links to documentation and source code ⁶⁹. Community support is available via mailing lists (`podman@lists.podman.io` which may migrate to CNCF lists) and forums like GitHub Discussions and IRC/Discord channels.

Podman on Alpine Linux

Alpine Linux – known for its lightweight and security-oriented design – supports Podman in its official package repositories. **Podman is included in Alpine's Community repository** on all current versions

of Alpine. For example, Alpine 3.18 ships Podman 4.5.1, and Alpine Edge (development branch) as of March 2026 includes Podman 5.8.1 ⁷⁰ ⁷¹. This means Alpine users can install Podman with a simple `apk add` command (see next section for detailed steps). The Alpine Linux Wiki explicitly notes: “Podman can be installed via the `podman` package in the community repository” ⁷². Because Podman is in Community, you may need to ensure the **community repository is enabled** in your `/etc/apk/repositories` (for stable Alpine releases, the community repo is typically commented out by default – uncomment the line for your Alpine version to gain access to community packages).

When you install Podman on Alpine, a number of ancillary packages come along to support container operations. The Podman package’s dependencies include: an OCI runtime (via a virtual `oci-runtime` package provided by **crun** by default on Alpine), **Conmon** (container monitor), **shadow-subids** (for subuid/subgid mapping in rootless mode), **iptables** (for networking rules), **Netavark** and **Aardvark-dns** (new networking backend and DNS for rootless networking), and **Catatonit** (a tiny init process for containers) ⁷³. These will be pulled in automatically. Alpine’s package maintainers have configured Podman in a way that is sensible for the distro’s environment – for instance, it prefers `crun` over `runc` for better rootless support, and it uses the **OpenRC init system** for any needed services (since Alpine doesn’t use systemd). There is a subpackage called `podman-openrc` which provides an OpenRC script to manage a Podman system service (for the API socket) ³². If you want Podman’s REST API or socket-activated service, you should install `podman-openrc` and enable the service. Similarly, Alpine provides `podman-docker` (to link the Docker CLI to Podman) and `podman-compose` (a Python tool to run Docker Compose files with Podman) as separate packages ⁷⁴. Documentation is split out into `podman-doc` – installing this will give you man pages and additional docs on the system ³².

Podman usage on Alpine is largely the same as on any other Linux, but there are a few Alpine-specific notes:

- **Cgroups:** Alpine uses OpenRC, so cgroup v2 is not automatically mounted on boot unless the `cgroups` service is enabled. The Alpine Wiki advises enabling the cgroups service (in *unified* mode) to ensure Podman has full functionality ⁵⁹. This is because Podman (and container engines generally) need the cgroup filesystem mounted to limit resources and track processes. We cover how to do this in the installation guide below.
- **Rootless Operation:** Alpine supports rootless Podman well, but you must configure subordinate UIDs and GIDs for your user if not already set. Typically, this means adding a line for your user in `/etc/subuid` and `/etc/subgid` (Alpine’s `adduser` might not do this by default). We will illustrate this in the guide. Also, for rootless networking, Podman will use `slirp4netns` by default (with the `tun` kernel module) or `passt` if installed. The wiki suggests loading the `tun` module and configuring it to auto-load to avoid issues with rootless networking ⁷⁵.
- **Storage Driver:** The default storage driver on Alpine is overlayfs (which requires the Alpine kernel to have overlay support – it does). If you run Podman **inside** an LXC container or Docker container (for example, using Alpine as a base image to run Podman – a “Podman in Docker” scenario), you might need to switch to the `vfs` driver because overlay cannot nest by default ⁶³. This can be done by editing `/etc/containers/storage.conf` or setting environment variable `STORAGE_DRIVER=vfs` for that session. Similarly, if Alpine’s filesystem is Btrfs, you could switch to the `btrfs` storage driver for efficiency ⁶³. These are edge cases, but worth knowing.
- **Alpine Kernel Limitations:** Alpine uses the **musl libc** and its kernels are often optimized for size. Most Podman features work fine, but if you encounter an issue (for example, certain

unprivileged overlay mount errors or seccomp differences), ensure your Alpine kernel is up-to-date. The Alpine wiki's troubleshooting section covers a few common problems, such as ensuring the root filesystem mount is `shared` if using tools like Distrobox ⁷⁶ ⁷⁷. A “`/ is not a shared mount`” error can occur when using Podman in combination with containerized dev environments – the wiki shows how to resolve that by marking the root mount as shared via OpenRC local scripts ⁷⁸ ⁷⁹. For typical use of Podman, this isn't needed unless you use Distrobox or similar (which create containers that need access to the host's mounts). The point is that Alpine can run Podman fully, but being a smaller distro, you might occasionally adjust configurations that are default in Fedora/Debian.

Finally, **Alpine's inclusion of Podman** indicates its growing popularity even in minimal distros. Alpine's maintainers regularly update the Podman package – for instance, Podman 5.x was packaged in Alpine edge within a short time of upstream release (e.g., Podman 5.3.2 was built in Jan 2025 ⁸⁰, and 5.8.1 in Mar 2026 ⁷⁰). The package is maintained by Alpine community members (the maintainer for recent versions is listed as Michał Polański ⁸⁰). They also provide an **APKBUILD** in Alpine's Git repository if one wants to see how Podman is built (link available on the Alpine packages site) ⁸¹. Overall, installing and using Podman on Alpine is straightforward, as we'll detail next.

Official Documentation and Resources

Users of Podman should take advantage of the rich documentation available both upstream and from distributions:

- **Podman Official Documentation:** The main docs are hosted on podman.io and the docs.podman.io site. This includes tutorials, a “getting started” guide ⁸², and full manual pages for every Podman command. For example, the `podman(1)` man page (which covers the CLI usage and options) can be read online ⁸³. There are also specific docs on rootless mode, configuration files, and integration with other tools. The upstream documentation is very comprehensive and is updated with each release.
- **Man Pages:** If you install the `podman-doc` package on Alpine, you will get local man pages for Podman commands ³². You can then use `man podman`, `man podman-run`, etc., on your Alpine system. Upstream man pages (in HTML form) are also available on the Podman docs site. For instance, docs.podman.io/en/latest/markdown/podman-run.1.html would show the manual for the `podman run` command. Another useful piece of documentation is the **Podman Troubleshooting Guide** on the Podman site and the **Podman FAQ** – these address common issues and differences from Docker.
- **Alpine Linux Wiki and Package Info:** Alpine's wiki has a dedicated page for Podman ⁸⁴, which provides Alpine-specific setup tips (much of which we've incorporated above). It's a great resource for Alpine users, covering configuration (cgroups, rootless, etc.) and troubleshooting on Alpine. Additionally, the Alpine package database online provides details on the Podman package: version, maintainer, build time, and dependencies ⁶⁹ ⁷³. It doesn't read like documentation, but if you're curious about exactly which versions are in each Alpine release or what subpackages exist, the pkgs.alpinelinux.org page is very handy. For example, you can see that the Podman package provides the `podman` and `podmansh` commands and is built from the upstream podman.io project ⁶⁹ ⁸⁵. The Alpine package's Git repository link (provided on that page) can be browsed to see Alpine-specific patches or build configurations if needed.

- **Upstream Project Links:** The source code for Podman is on GitHub at [containers/podman](#) ⁸⁶. There, you can find the README, contribution guidelines, release notes, and issues. The GitHub wiki also has additional info (though most official docs have moved to the [podman.io](#) site). For learning and community interaction, Podman has an IRC channel on Libera ([#podman](#)), a Discord server (Containers community), and mailing lists (which are moving under CNCF's lists). The [Podman community page](#) links to these resources.
- **Related Tools Documentation:** If your use of Podman extends to building images or handling registries, note that Podman often works in tandem with Buildah and Skopeo. They are separate tools but share a lot. Buildah is for building OCI images (and Podman actually calls Buildah's functionality internally for `podman build`) ³⁵. Skopeo lets you copy images between registries or to/from storage. Documentation for these is also on Podman's site (or their respective GitHub repos). The reason to mention this is if you dive deep into Podman, you'll encounter these names, and their official docs are worth reading for advanced image management tasks.
- **Distribution Docs:** Aside from Alpine, other distros have Podman docs which can be informative. For example, Fedora, Red Hat, and SUSE all have sections in their knowledgebase or docs about Podman usage. Red Hat's docs (for RHEL) and the Red Hat Developer blog have numerous articles on Podman (like *"3 advantages of Podman vs Docker"* ⁸⁷, tutorials on rootless Podman, etc.). SUSE's documentation also outlines how to run Podman rootless and is relevant because it's another non-systemd environment case ⁶⁰. These external resources can provide different perspectives or troubleshooting advice.
- **Manuals and Books:** An entire book *"Podman in Action"* (by Daniel Walsh and Scott McCarty, Manning Publications) was published, and while not free, it indicates the level of depth Podman has achieved. The Podman community wiki (in GitHub) lists some recordings of webinars and conference talks too, which can be educational (e.g., *"Understanding Podman Internals"* on YouTube ⁸⁸).

In summary, users should refer to [podman.io](#) for official guides and manuals, the **Alpine Wiki** for Alpine-specific notes ⁸⁹, and consider installing the `podman-doc` package for on-system help. Always ensure the documentation version corresponds to your Podman version (Podman has evolved quickly, and features in 4.x or 5.x might not apply if running an older version in an LTS distro).

Below, we provide a detailed installation guide for Podman on Alpine Linux, incorporating the points discussed:

Installation Guide: Podman on Alpine Linux

This step-by-step guide will help you install and configure Podman on Alpine Linux. It is written sequentially so that it can be easily followed or adapted into a shell script. We will cover enabling the repository, installing Podman and related packages, configuring the system (cgroups, subuids for rootless, etc.), and verifying that Podman works.

Prerequisites: You should have Alpine Linux (3.17 or later recommended) with root access (to install packages and adjust configs). If on a standard Alpine install, ensure you have network connectivity to fetch packages.

1. Enable the Alpine Community Repository (if not already enabled). Podman resides in the *community* repo on Alpine. Open the repositories file (e.g. `/etc/apk/repositories`) and make sure the line for your Alpine version's community repo is present and not commented out. For example, you might uncomment a line like:

```
# nano /etc/apk/repositories
```

```
https://dl-cdn.alpinelinux.org/alpine/v3.18/community
```

If you're on Alpine Edge, use the `edge/community` repo. Many Alpine installations have this enabled by default, but it's good to check. *(If it was commented and you uncomment, run `apk update` afterward.)*

2. Update package index and install Podman. Use Alpine's package manager `apk` to fetch the latest package lists and install Podman:

```
apk update
apk add podman podman-openrc podman-doc
```

This will install the Podman engine and its dependencies. In the command above, we also included: - `podman-openrc` - which provides the OpenRC init script for Podman (allowing you to start a Podman service for the REST API if needed). - `podman-doc` - which installs documentation files like man pages (optional but recommended for offline help).

After this, you should have the `podman` command available. You can verify by checking the version:

```
podman --version
```

This should output Podman's version (e.g., *podman version 5.3.2*). At this point, running `podman info` will likely show some warnings if cgroups aren't set up; we'll address those next.

3. Enable cgroups (control groups) in unified mode. Alpine uses OpenRC, and by default, cgroup v2 might not be mounted. Podman requires cgroup support for managing resources and should use cgroup **v2** (unified hierarchy) for full functionality ⁵⁹. To enable this, run:

```
rc-update add cgroups default
rc-service cgroups start
```

This registers the `cgroups` service to start at boot in the *default* runlevel and immediately starts it now. The service will mount the cgroup filesystem (typically at `/sys/fs/cgroup`) in *unified* (v2) mode, which is the default configuration on Alpine 3.14+ ⁵⁹. After running this, you can check `mount | grep cgroup` to see that **cgroup2** is mounted. If you reboot later, the cgroups will be set up automatically on boot.

4. Configure rootless container support (optional, for running Podman as non-root). If you plan to run containers as a **regular user** (which is a major advantage of Podman), perform these steps. They can be skipped if you only intend to use Podman as root (in which case, jump to step 5).

a. **Subuid/Subgid mapping:** Ensure your user has subordinate UID and GID ranges allocated. If you installed `shadow-subids` (it comes with Podman by default on Alpine ⁵⁴), you have the tools needed. Now edit (or create) the files `/etc/subuid` and `/etc/subgid`. Add a line for your username with a UID/GID range. For example, for user `"alice"`:

```
echo "alice:100000:65536" | tee -a /etc/subuid /etc/subgid
```

This gives user `alice` a block of 65,536 UIDs (100000–165535) to use in user namespaces ⁵³. You can choose a different start ID if that one is taken; just ensure the format is `username:startid:count` and that IDs don't overlap between users. By default, many distros allocate 65536 UIDs per user. The specific numbers aren't critical as long as they're unique per user.

b. **Load kernel module for rootless networking:** Podman rootless uses `slirp4netns` which requires the `tun` device (for setting up a user-space NAT network). Alpine's wiki suggests loading the `tun` kernel module and persisting it:

```
modprobe tun
echo "tun" >> /etc/modules
```

This will load the `tun` module now and ensure it loads on each boot (the `/etc/modules` file is read by OpenRC to preload modules). If `tun` is not available, rootless containers will have no networking or will fail to start with network errors. The above ensures the virtual TUN/TAP device support is ready ⁹⁰.

c. *(Optional)* **Use crun for rootless runtime:** Alpine's Podman by default should pull in `crun`, which is a better OCI runtime for rootless mode (it's faster and handles cgroup2 well). You can verify by running `podman info | grep OCI\ runtime`. If it shows `crun` in the list and default, you're good. If it's using `runc` and you encounter permission issues, consider installing `crun` (`apk add crun`) and updating Podman's config to use it. In many cases, this isn't necessary as the dependency is already satisfied by `crun` ⁵⁶ ⁵⁷.

d. After these changes, it's a good idea to **re-log or reboot** for them to take effect for your user. Logging out and back in (or using `newuidmap` manually) will ensure your user session recognizes the subuid mappings.

5. (Optional) Configure Podman socket service (for Docker-compatible API). If you need to use tools that speak the Docker API (for example, Docker Compose, or an IDE expecting a Docker socket), you can start the Podman service which provides a REST API at `/run/podman/podman.sock`. On Alpine, since we installed `podman-openrc`, do:

```
rc-service podman start
```

This starts the Podman RESTful service daemon (rootful by default). It listens on the Unix socket `/run/podman/podman.sock` ³¹. To have this start on boot:

```
rc-update add podman default
```

Now, keep in mind, Podman's socket is *not* identical to Docker's default socket path. Some tools might look for `/var/run/docker.sock`. You can either configure them to use `/run/podman/podman.sock` or create a symlink for convenience (`ln -s /run/podman/podman.sock /var/run/docker.sock`). **Security note:** The Podman socket grants similar abilities as Docker's socket would (container root access). It's advisable not to open this socket to non-admin users or over the network. In rootless mode, Podman can also serve a user-specific socket (`$XDG_RUNTIME_DIR/podman/podman.sock`), which would require launching the service as the user (e.g., `podman system service --unix=/tmp/podman.sock`). This is advanced usage; most users don't need the API and can skip running the service.

6. Verify Podman is working (smoke test). Now it's time to test a simple container. You can do this as root or as a regular user (if you configured rootless). For example, try running the classic hello-world container:

```
podman run --rm hello-world
```

The first run will trigger Podman to pull the `hello-world` image from Docker Hub (or another default registry). Since this is the first container, Podman will set up storage and the network. You should see output from the container confirming it ran (the hello-world container prints a greeting message). Podman will then remove the container (`--rm` means remove after running). If this works, congratulations – Podman is up and running on Alpine! ⁹¹ ⁹²

If it failed, check common issues: - **Missing subuid mappings:** If you got an error about inability to use user namespace, double-check `/etc/subuid` and `/etc/subgid` for your user and ensure you started a new shell after editing them. Also verify `shadow-subids` installed the `newuidmap`/`newgidmap` binaries (which it should have, in `/usr/bin`). These are needed for Podman to create the user namespace. - **Cgroups not mounted:** If you see errors related to cgroups or “did not receive systemd slice”, make sure step 3 was done and that `/sys/fs/cgroup` shows a unified hierarchy (you can run `podman info | grep -A5 "cgroupVersion"` to see if it says v2). If not, a reboot after enabling cgroups might fix it, or there may be an issue with the cgroups service (ensure your kernel supports cgroup v2). - **Networking issues:** If the container couldn't reach the internet or you got `cannot open slirp4netns` errors in rootless mode, ensure the `slirp4netns` package is installed (it should have been, via Podman depends). You can test by running `slirp4netns --version`. Also ensure the `tun` module is loaded as in 4b. For rootful containers, Podman uses CNI – check that `iptables` is present (it was installed as a dependency) and that the `cni0` (or `podman0`) interface was created (`ip a` should show it after running a container). - **Storage driver issues:** If you are running on a filesystem that doesn't support overlay (or inside Docker), and you see errors about overlay mounts, switch to vfs driver as mentioned earlier (edit `/etc/containers/storage.conf`: set `driver = "vfs"` under `[storage]`). This will eliminate overlay but at the cost of performance and space (no copy-on-write). For casual testing it's fine.

7. (Optional) Using Podman as a Docker replacement in scripts: If you have existing Docker-based scripts or Compose files: - Install `podman-docker` (via `apk add podman-docker`). This package will install a script that provides a `/usr/bin/docker` command forwarding to Podman ²⁰. It allows many Docker-oriented tools to work transparently with Podman. Keep in mind not everything is perfectly replicated (for example, `docker logs -f` works with Podman, but `docker swarm`

commands obviously have no Podman equivalent). - For **Docker Compose**: Alpine has a package `podman-compose` (a Python tool) ⁹³. You can `apk add podman-compose` which allows you to do `podman-compose up` with your existing `docker-compose.yml`. There's also a newer Compose v2 plugin for Podman (`podman compose` command built-in). In Podman 4.1+, you can run `podman compose up` (note: no hyphen in subcommand) using the builtin go-based compose. Check Podman's version – if it's new enough, you might prefer the built-in compose support. Either way, Podman aims to run Docker Compose files with minimal changes (just ensure the service definitions are compatible and be mindful of any Docker-specific features).

8. (Optional) Running on boot and auto-updates: If you want certain containers or pods to start at boot on Alpine, the simplest way is to use Podman's generated systemd units. However, Alpine doesn't use systemd by default. You have a few choices: - Use the `podman-openrc` service approach for *specific containers*: For each container you want running, you could create an OpenRC local service that calls `podman run` or `podman start`. E.g., write an init script or use `/etc/local.d` scripts to start containers (remember to `podman generate systemd` in a chroot if needed to get the commands). This is a bit manual on Alpine. - Use **crontab @reboot** or an OpenRC custom service as a lightweight way to start containers. - If you switch Alpine to use s6 or another init that can import systemd units (less common), or if you use Dinit/OpenRC mix, you could explore Podman's auto-update and healthcheck features. Podman has a label-based auto-update mechanism that, in conjunction with systemd timers, can auto pull new images and restart containers. On Alpine, implementing this requires some custom setup since systemd is not present.

For most Alpine users, step 8 might be overkill; they'll run containers as needed or manage them via scripts.

9. Keep Podman updated: Alpine will update Podman in its stable releases via patch versions (security fixes, minor version bumps) and major updates come with new Alpine releases. Use `apk upgrade` regularly to get the latest fixes. Podman's development is active, so staying updated is wise especially for security patches or bug fixes (e.g., a fix for an OCI runtime error or CVE). You can check the Alpine package changelog via the git commit history on Alpine's GitLab (link is on the package page) ⁸¹ for any notable changes between versions.

After completing these steps, you have a fully functioning Podman setup on Alpine Linux. You can now enjoy managing containers without a daemon and with the enhanced security of rootless mode. For more detailed usage, refer to Podman's manuals (e.g. `man podman`, or the official docs) and the Alpine Wiki if you run into Alpine-specific quirks. Happy containering with Podman!

17 89

1 8 65 68 86 Podman - Wikipedia

<https://en.wikipedia.org/wiki/Podman>

2 4 9 10 11 12 13 14 18 19 21 23 24 37 38 39 49 51 52 55 64 Docker vs Podman vs containerd – Which Container Runtime to Choose in 2026 | EITT

<https://eitt.academy/knowledge-base/docker-vs-podman-vs-containerd-comparison-2026/>

3 5 16 28 34 42 43 44 45 Podman vs Docker: Complete 2026 Comparison Guide for DevOps Teams | Xurrent Blog

<https://www.xurrent.com/blog/podman-vs-docker-complete-2025-comparison-guide-for-devops-teams>

6 40 41 66 Exploring Cloud Native projects in CNCF Sandbox. Part 5: 13 arrivals of January 2025 | Tech blog | Palark

<https://palark.com/blog/cncf-sandbox-2025-jan/>

7 podman - Red Hat Ecosystem Catalog

<https://catalog.redhat.com/en/software/containers/rhel9/podman/618412b6f286970f08bf4a36>

15 22 25 26 27 36 48 Docker vs. Podman vs. Containerd vs. nerdctl: 2025 Showdown | sanj.dev

<https://sanj.dev/post/docker-vs-podman-comparison>

17 29 30 35 50 83 podman — Podman documentation

<https://docs.podman.io/en/stable/markdown/podman.1.html>

20 32 54 61 69 73 74 80 81 85 podman - Alpine Linux packages

https://pkgs.alpinelinux.org/package/edge/community/x86_64/podman

31 53 58 59 63 72 75 76 77 78 79 84 89 90 91 92 93 Podman - Alpine Linux

<https://wiki.alpinelinux.org/wiki/Podman>

33 5 Podman features that can benefit your IT architecture - Red Hat

<https://www.redhat.com/en/blog/podman-container-intro>

46 47 87 3 advantages of Podman vs. Docker | Red Hat Developer

<https://developers.redhat.com/articles/2023/08/03/3-advantages-docker-podman>

56 57 crun - Alpine Linux packages

https://pkgs.alpinelinux.org/package/edge/community/x86_64/crun

60 Running Podman in Rootless Mode - SUSE Documentation

<https://documentation.suse.com/smart/container/html/rootless-podman/index.html>

62 82 Podman Installation | Podman

<https://podman.io/docs/installation>

67 MAINTAINERS.md - Podman.io - GitHub

<https://github.com/containers/podman.io/blob/main/MAINTAINERS.md>

70 Package index - Alpine Linux packages

https://pkgs.alpinelinux.org/packages?name=podman&branch=edge&repo=&arch=x86_64&maintainer=

71 podman - Alpine Linux packages

<https://pkgs.alpinelinux.org/packages?name=podman&branch=v3.18>

88 Understanding the Podman Internals - YouTube

<https://www.youtube.com/watch?v=kJnxeinEWyA>